

Capítulo 1 — Del navegador al servidor: el sistema de nombres de dominio

Unidad de estudio · Edición ampliada con fundamentos teóricos

1.1. Alcance de la clase

Este capítulo cubre el único tramo del despliegue que no ocurre en el servidor ni en el código: la traducción de un nombre legible en una dirección de red. Es también el tramo donde se concentra la mayor cantidad de fallos difíciles de diagnosticar, por una razón estructural que conviene enunciar desde el principio: **cuando la resolución de nombres falla, el servidor no registra absolutamente nada**. No hay línea de log que revisar, porque la petición nunca llegó a emitirse hacia él.

A diferencia de otros temas del módulo, el sistema de nombres de dominio no puede abordarse como una mera secuencia de pasos de configuración. Los pasos son pocos —cargar dos registros en un panel—, pero interpretarlos exige comprender una arquitectura distribuida que tiene más de cuarenta años de historia, un modelo de delegación jerárquica y un mecanismo de caché que gobierna todos los tiempos de espera observables. Por eso este capítulo intercala el fundamento teórico con el procedimiento: cada decisión práctica se apoya en un concepto que la explica, y cada concepto se ilustra de inmediato con su consecuencia operativa.

Al finalizar la clase, cada grupo debe contar con un nombre de dominio propio, sus registros cargados y la resolución verificada desde al menos dos servidores distintos.

Contenidos

1. Origen y objetivos de diseño del sistema de nombres de dominio.
2. El recorrido completo de una petición HTTPS.
3. Arquitectura del sistema de nombres: espacio de nombres, zonas y resolución.
4. Anatomía del mensaje DNS y sus códigos de respuesta.
5. Tipos de registro y sus restricciones.
6. Delegación mediante servidores de nombres.
7. Tiempo de vida, caché y el concepto erróneo de "propagación".
8. Estudio de caso: filtrado de dominios autogenerados por proveedores argentinos.
9. Herramientas de diagnóstico.
10. Seguridad y evolución del protocolo: DNSSEC, DoT y DoH.
11. Registros comodín: dos conceptos distintos bajo el mismo nombre.
12. Obtención de un dominio mediante el GitHub Student Developer Pack.
13. Carga y verificación de los registros.

1.2. Por qué existe un sistema de nombres: origen y diseño del DNS

Las redes de computadoras funcionan con direcciones numéricas. Una dirección IP identifica una interfaz de red con precisión absoluta, pero es hostil para la memoria humana y, peor aún, es inestable: un servicio puede mudarse de servidor, cambiar de proveedor o repartirse entre varias máquinas, y en todos esos casos su dirección cambia. Desde los orígenes de las redes se necesitó una capa de indirección entre el nombre con el que las personas se refieren a un servicio y la dirección con la que las máquinas lo alcanzan. Esa capa es, conceptualmente, lo mismo que una agenda de contactos: uno recuerda "Ana" y la agenda resuelve el número de teléfono; si Ana cambia de número, el nombre sigue siendo válido y solo se actualiza la entrada.

La primera implementación de esa agenda fue literal. En ARPANET, la red precursora de internet, existía un único archivo de texto llamado `HOSTS.TXT`, mantenido a mano por el Network Information Center del Stanford Research Institute. Cada administrador que conectaba una máquina nueva enviaba el alta por correo electrónico; el centro actualizaba el archivo, y todas las computadoras de la red lo descargaban periódicamente por FTP. El esquema funcionó mientras la red tuvo unos cientos de nodos, y colapsó exactamente por las razones que uno esperaría de cualquier lista centralizada y copiada a mano: el archivo crecía sin límite, el tráfico para descargarlo se multiplicaba, los nombres empezaron a colisionar porque no había autoridad que garantizara unicidad más allá de una oficina, y la copia que cada máquina tenía estaba siempre desactualizada respecto de la realidad. El vestigio de ese archivo sobrevive en todos los sistemas operativos actuales: `/etc/hosts` en Unix y `C:\Windows\System32\drivers\etc\hosts` en Windows siguen existiendo, y siguen teniendo prioridad sobre cualquier consulta a la red.

En 1983, Paul Mockapetris propuso el reemplazo en las RFC 882 y 883, refinadas en 1987 en las RFC 1034 y 1035, que siguen siendo la base normativa del protocolo. El **Domain Name System** resolvió el problema con tres decisiones de diseño que explican todo lo que se observa en la práctica de este capítulo.

Primera decisión: distribuir la autoridad, no los datos. En lugar de un archivo único, el DNS define un espacio de nombres jerárquico donde cada nivel puede delegar la administración de sus subniveles a organizaciones distintas. Nadie posee la base de datos completa; cada organización es autoritativa únicamente sobre su porción, y la unicidad de los nombres queda garantizada por construcción: dentro de `utn.edu.ar`, la universidad decide los nombres sin consultar a nadie, y ningún externo puede crear nombres allí.

Segunda decisión: aceptar la coherencia eventual a cambio de escala. El DNS renuncia deliberadamente a que todos los participantes vean el mismo dato en el mismo instante. Cada respuesta lleva un tiempo de vida (TTL) durante el cual puede ser copiada y reutilizada por intermediarios. El resultado es un sistema donde el 99% de las consultas se responde desde una copia cercana sin tocar al servidor de origen, al precio de que un cambio tarde en ser visto por quienes guardaron la copia anterior. Este compromiso —disponibilidad y escala por encima de consistencia instantánea— es el mismo que estudiarán después en bases de datos distribuidas, y es la clave de la sección 1.8.

Tercera decisión: un esquema de datos extensible. El DNS no asocia nombres solamente con direcciones IP: asocia nombres con **registros de recursos** tipados. Un mismo nombre puede tener a la vez una dirección IPv4, una dirección IPv6, un servidor de correo y un texto de verificación de

titularidad, cada uno en un registro de tipo distinto. Esta generalidad es la razón por la que el DNS sobrevivió cuatro décadas de cambios tecnológicos: cuando apareció IPv6 se agregó el tipo AAAA sin tocar el protocolo, y lo mismo ocurrió con los tipos de seguridad que se estudian en la sección 1.11.

🔦 PARA ENTENDER

La moraleja histórica no es anecdótica: todo sistema centralizado de nombres colapsa por escala, y todo sistema distribuido con caché paga el precio de la coherencia eventual. El DNS eligió lo segundo. Cada vez que en este práctico "un cambio no se ve" o "a un compañero le anda y a otro no", están observando esa decisión de diseño de 1983, no un error.

1.3. Qué ocurre entre la barra de direcciones y la respuesta

El acto de escribir `https://calculadora.tudominio.com` y presionar Enter desencadena cinco operaciones sucesivas e independientes entre sí. Distinguir las es la base de todo diagnóstico posterior, porque cada una puede fallar por causas propias, deja (o no deja) rastros en lugares distintos y se repara con herramientas distintas. La ingeniería de redes se estudia siempre así, por capas: cada operación consume el resultado de la anterior y no sabe nada de las que siguen.

#	Operación	Qué se obtiene	Quién interviene
1	Resolución de nombre	Una dirección IP	Resolver DNS
2	Conexión TCP	Un canal abierto al puerto 443	Red, firewall del servidor
3	Negociación TLS	Un canal cifrado y un certificado validado	Servidor web, autoridad certificadora
4	Petición HTTP	El recurso solicitado	Proxy inverso y aplicación
5	Respuesta	El contenido renderizado	Navegador

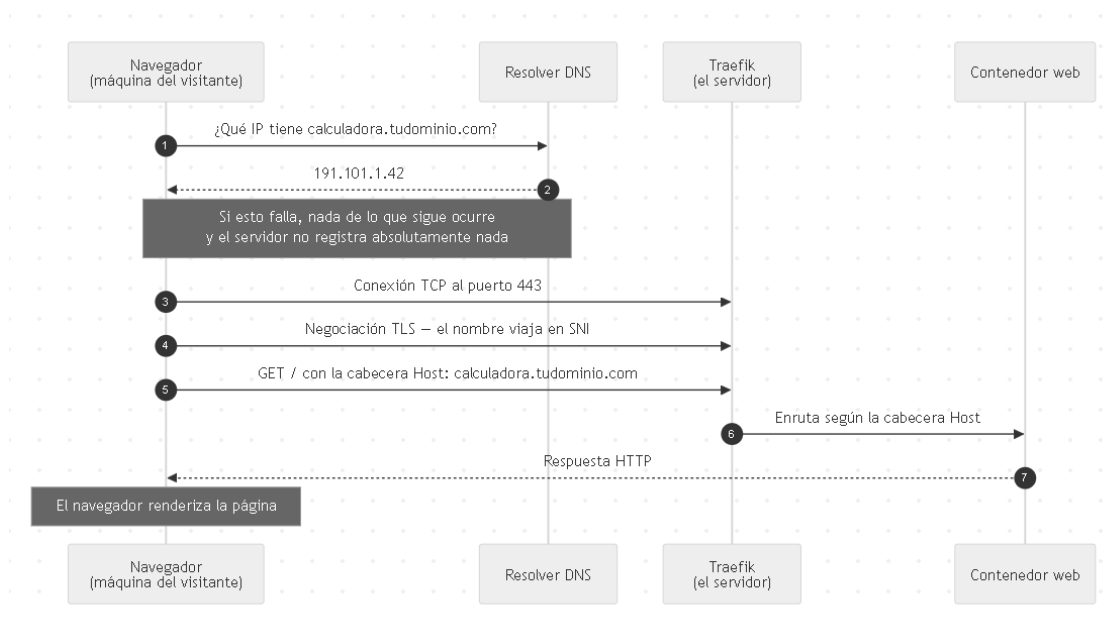


Figura 1.1. Recorrido de una petición HTTPS. La resolución de nombres es previa e independiente de la conexión con el servidor.

Tres observaciones sobre este esquema, que se retoman a lo largo de todo el módulo.

La resolución es previa e independiente de la conexión. El sistema de nombres no establece ninguna comunicación con el servidor de destino: se limita a devolver una dirección. Si la operación 1 falla, las cuatro restantes no se ejecutan. Esta independencia tiene una consecuencia probatoria que conviene fijar como método de trabajo: el log del servidor solo puede atestiguar sobre las operaciones 2 a 5. Un servidor con el log vacío no está "sin actividad": está ciego respecto de un fallo que ocurre antes de su puerta.

El nombre solicitado viaja dentro de la petición. En la operación 4, el navegador incluye una cabecera Host con el nombre que el usuario escribió:

```
GET / HTTP/1.1
Host: calculadora.tudominio.com
```

Esta cabecera es el mecanismo que permite que un único servidor, con una única dirección IP, atienda múltiples dominios distintos. El proxy inverso lee ese valor y decide a qué servicio interno entregar la petición. La técnica se denomina **alojamiento virtual** (virtual hosting) y es históricamente importante: sin ella, cada sitio web del mundo necesitaría una dirección IP exclusiva, algo imposible dada la escasez de direcciones IPv4. Es la que hace posible que el VPS del práctico aloje tanto `calculadora.tudominio.com` como `api.tudominio.com` con una sola IP.

El nombre también viaja antes del cifrado. Durante la negociación TLS, el navegador declara el nombre solicitado mediante la extensión SNI (*Server Name Indication*), para que el servidor sepa qué certificado presentar. Esto ocurre antes de que exista canal cifrado alguno, porque el certificado es precisamente el material con el que se establecerá ese canal: hay un problema de huevo y gallina que la extensión SNI resuelve enviando el nombre en claro. Es un detalle relevante en la Clase 4, cuando cada servicio obtenga su propio certificado.

PARA ENTENDER

Fijate en la consecuencia práctica de que los pasos 1 y 2 sean independientes. Si el DNS falla, en el servidor **no hay nada que mirar**. El log está vacío, el servicio está sano, y todo funciona perfecto. Cuando algo no te ande y el servidor no diga nada, el DNS es el primer sospechoso, no el último.

1.4. Arquitectura del sistema de nombres

El sistema de nombres de dominio no es una base de datos centralizada. Es una jerarquía distribuida cuya estructura lógica conviene entender antes de enumerar sus actores, porque el procedimiento de carga de registros de la sección 1.14 es una consecuencia directa de esta estructura.

1.4.1. El espacio de nombres como árbol

Formalmente, el espacio de nombres del DNS es un árbol invertido. En la cima hay un nodo raíz sin nombre; de él cuelgan los dominios de primer nivel (*Top-Level Domains* o TLD) como `com`, `ar`, `me` o `tech`; de cada TLD cuelgan los dominios registrables como `tudominio.com`; y de cada dominio pueden colgar tantos subdominios como su titular decida crear. Un nombre de dominio completo no es más que la lectura del camino desde una hoja hasta la raíz, con las etiquetas separadas por puntos: `calculadora.tudominio.com` se lee "el nodo `calculadora`, hijo de `tudominio`, hijo de `com`, hijo de

la raíz". Por eso los nombres se interpretan de derecha a izquierda: la parte más significativa —la que determina quién tiene autoridad— es la última.

El estándar impone límites concretos que ocasionalmente aparecen en la práctica: cada etiqueta admite hasta 63 caracteres, y el nombre completo hasta 255. Un nombre que termina explícitamente con un punto final —`calculadora.tudominio.com.`— se denomina **nombre completamente calificado** (FQDN, *Fully Qualified Domain Name*): el punto final representa a la raíz y elimina toda ambigüedad. La mayoría de las herramientas lo agregan de forma implícita, pero algunos paneles DNS lo muestran, y verlo no debe interpretarse como un error.

Sobre este árbol se define la distinción más importante del capítulo: la diferencia entre **dominio** y **zona**. Un dominio es un subárbol completo: el dominio `com` incluye, en sentido amplio, a todo lo que termina en `.com`. Una zona, en cambio, es la porción del árbol que una organización administra efectivamente, y termina exactamente donde comienza una delegación hacia otra organización. El operador de `com` no administra los registros de `tudominio.com`: su zona contiene apenas la anotación de a quién le delegó ese subárbol. Los registros concretos viven en la zona delegada, administrada por el proveedor que el titular eligió. La delegación es el acto administrativo que corta el árbol en zonas, y es el tema de la sección 1.7.

1.4.2. Los tres niveles de autoridad

En la práctica, la jerarquía se organiza en tres niveles de autoridad.

Nivel	Responsable	Qué sabe
Raíz	13 conjuntos de servidores raíz	Quién administra cada dominio de primer nivel
Primer nivel (TLD)	Operador de <code>.com</code> , <code>.ar</code> , <code>.me</code> , <code>.tech</code>	Qué servidores administran cada dominio registrado
Autoritativo	El proveedor donde el titular administra su zona	Los registros concretos del dominio

Sobre los servidores raíz vale una precisión que suele generar confusión: los "13 servidores" no son 13 máquinas. Son 13 identidades lógicas —`a.root-servers.net` hasta `m.root-servers.net`—, un límite heredado del tamaño máximo que podía tener una respuesta DNS sobre UDP en el diseño original. Cada identidad está replicada hoy en cientos de instancias físicas distribuidas por el mundo mediante **anycast**, una técnica de enrutamiento en la que muchas máquinas comparten la misma dirección IP y la red entrega cada consulta a la instancia más cercana. Hay instancias de servidores raíz en América del Sur, incluida Argentina: consultar "la raíz" no implica cruzar el océano.

1.4.3. Resolución recursiva e iterativa

Ninguno de estos niveles es consultado directamente por el navegador. Entre el usuario y la jerarquía se interpone un cuarto actor, que es el que realiza el trabajo: el **resolver recursivo**.

La separación de roles es precisa y tiene nombres técnicos. El equipo del usuario ejecuta apenas un **stub resolver**: una pieza mínima del sistema operativo que sabe hacer una única cosa, enviarle la pregunta completa a un resolver recursivo y esperar la respuesta final. Esa consulta se llama **recursiva** porque el stub le pide al resolver que se haga cargo de todo el recorrido. El resolver recursivo, a su

vez, interroga a la jerarquía mediante consultas **iterativas**: le pregunta a un servidor raíz, que no conoce la respuesta final pero responde con una **referencia** ("no sé, pero los servidores de `.com` son estos"); repite la pregunta al servidor de `.com`, que responde con otra referencia hacia los servidores autoritativos del dominio; y finalmente pregunta al autoritativo, que responde con el dato. El resolver consolida ese recorrido en una única respuesta, la guarda en su caché y la devuelve al stub.

Un servidor autoritativo y un resolver recursivo cumplen funciones opuestas y conviene no mezclarlas: el autoritativo **posee** los datos de su zona y responde solo sobre ella, sin salir a preguntar nada; el recursivo no posee ningún dato propio, pero sabe **encontrar** cualquiera. El resolver que un equipo utiliza por defecto es el que le asigna su proveedor de internet mediante DHCP, aunque puede reemplazarse por uno público.

Resolver	Dirección	Operador
Predeterminado	Variable	El proveedor de internet
Google Public DNS	8.8.8.8	Google
Cloudflare	1.1.1.1	Cloudflare
Quad9	9.9.9.9	Quad9 (con filtrado de amenazas)

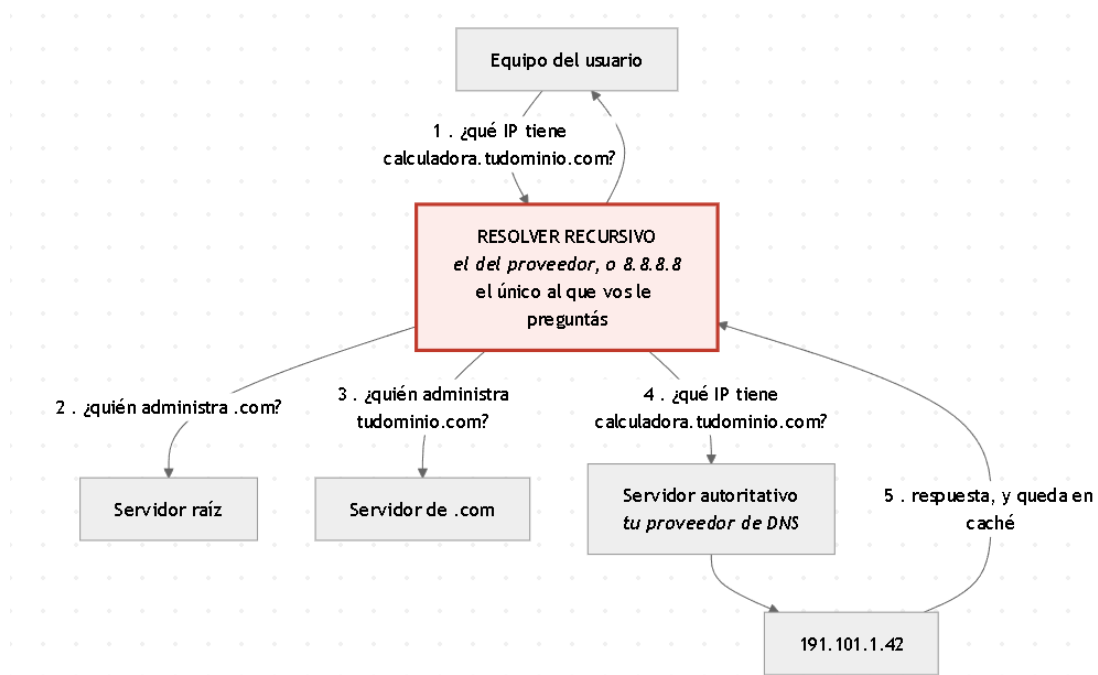


Figura 1.2. Resolución recursiva. El usuario nunca consulta la jerarquía directamente: siempre lo hace a través de un resolver.

La secuencia completa para resolver `calculadora.tudominio.com` es la siguiente:

1. El navegador consulta al resolver configurado en el sistema operativo.
2. El resolver consulta a un servidor raíz: *¿quién administra `.com`?*
3. El resolver consulta al servidor de `.com`: *¿quién administra `tudominio.com`?*

4. El resolver consulta al servidor autoritativo: *¿cuál es la dirección de `calculadora.tudominio.com`?*
5. El resolver almacena la respuesta en su caché y la devuelve al navegador.

En la práctica, los pasos 2 y 3 casi nunca ocurren: el resolver de un proveedor grande responde millones de consultas por hora y tiene en caché las referencias de la raíz y de todos los TLD frecuentes desde hace mucho tiempo. El recorrido completo solo se ejecuta para la porción del nombre que el resolver no conoce todavía. Esta es la primera manifestación del principio de caché que gobierna la sección 1.8.

PARA ENTENDER

Acá está la idea que hay que llevarse de toda la sección, y es la que después explica el problema con Claro: **vos nunca le preguntás al dominio. Le preguntás a un resolver.** Y ese resolver es un intermediario que puede tener la respuesta vieja en caché, puede estar caído, o puede decidir directamente no contestarte. El DNS no es una verdad universal: es lo que te dice el que te atiende.

1.5. Anatomía de una consulta: el mensaje DNS

Hasta aquí se describió quién pregunta y quién responde. Esta sección examina brevemente **qué** se transmite, porque los códigos que viajan dentro del mensaje DNS son exactamente los que las herramientas de diagnóstico muestran, y saber leerlos convierte la salida de `nslookup` o `dig` de un texto críptico en un informe preciso.

Una consulta y su respuesta comparten el mismo formato, definido en la RFC 1035: una cabecera fija de 12 bytes seguida de hasta cuatro secciones de contenido. La cabecera lleva un identificador que permite aparear cada respuesta con su pregunta, y un conjunto de indicadores (*flags*) de un bit que describen la naturaleza del mensaje. Cuatro de ellos aparecen constantemente en el trabajo de diagnóstico: **QR** distingue pregunta de respuesta; **RD** (*Recursion Desired*) es la marca con la que el stub resolver pide "hacete cargo de todo el recorrido"; **RA** (*Recursion Available*) es la confirmación del resolver de que ofrece ese servicio; y **AA** (*Authoritative Answer*) indica que quien responde es el servidor autoritativo de la zona, es decir, que el dato viene del origen y no de una caché.

Las cuatro secciones de contenido son: la **pregunta** (el nombre y el tipo de registro consultado), la **respuesta** (los registros que contestan la pregunta), la sección de **autoridad** (los servidores autoritativos involucrados, que es donde viajan las referencias de la resolución iterativa) y la sección **adicional** (datos complementarios que ahorran consultas futuras).

La cabecera incluye además un **código de respuesta** (RCODE) que resume el desenlace de la consulta. Los cuatro valores relevantes para este práctico son los siguientes.

RCODE	Nombre	Significado exacto
0	NOERROR	La consulta se procesó correctamente. Atención: no garantiza que haya registros en la respuesta; un nombre que existe pero no tiene registros del tipo pedido también devuelve NOERROR, con cero respuestas.

RCODE	Nombre	Significado exacto
2	SERVFAIL	El servidor no pudo procesar la consulta: un fallo interno, un problema con la zona, o una validación de seguridad que no pasó.
3	NXDOMAIN	El nombre consultado no existe en la zona autoritativa (o eso afirma quien responde). Es el código central del estudio de caso de la sección 1.9.
5	REFUSED	El servidor se niega a responder por política: reconoce la consulta como válida pero decide no atenderla.

El transporte también explica síntomas observables. El DNS opera históricamente sobre UDP en el puerto 53, un protocolo sin conexión elegido porque el intercambio típico es un único datagrama de ida y otro de vuelta: establecer una conexión completa sería un costo desproporcionado. El diseño original limitaba las respuestas UDP a 512 bytes; cuando una respuesta no cabe, el servidor activa el indicador **TC** (*truncated*) y el cliente reintenta por TCP, también en el puerto 53. La extensión EDNS(0), definida en la RFC 6891, amplió ese límite y es hoy de uso universal, pero la convivencia UDP/TCP sigue vigente y es la razón por la que un firewall que bloquea "el puerto 53 TCP por las dudas" produce fallos intermitentes que solo afectan a respuestas grandes.

PARA ENTENDER

La distinción entre RCODEs no es una curiosidad de protocolo: es información de diagnóstico de primera calidad. **NXDOMAIN es una respuesta**, rápida y definitiva: alguien te está diciendo "ese nombre no existe". **Un timeout no es una respuesta**: nadie te contestó nada. En la sección 1.9 esta diferencia separa dos problemas totalmente distintos que a simple vista parecen el mismo.

1.6. Tipos de registro

La unidad de información del DNS es el **registro de recurso** (*Resource Record*, RR). Todo registro, sin importar su tipo, comparte la misma estructura de cinco campos: el **nombre** al que pertenece, el **tipo** que declara qué clase de dato contiene, la **clase** (en internet, siempre **IN**), el **TTL** que fija cuánto tiempo puede guardarse en caché, y los **datos** propiamente dichos, cuyo formato depende del tipo. Cuando en un panel se carga "un registro A con nombre *****, valor **72.61.33.27** y TTL de 5 minutos", se están completando exactamente esos campos; el panel no inventa nada, solo les pone etiquetas amigables.

Una **zona** DNS es, en definitiva, el conjunto de registros de recursos bajo una misma autoridad administrativa. Toda zona contiene obligatoriamente dos registros estructurales: el **SOA** (*Start of Authority*), que declara los parámetros administrativos de la zona —el servidor primario, el contacto responsable, un número de serie que crece con cada modificación y los temporizadores de sincronización entre servidores—, y los registros **NS**, que enumeran sus servidores autoritativos. El SOA rara vez se edita a mano en un panel comercial, pero uno de sus campos reaparece en este capítulo con nombre propio: el último parámetro del SOA gobierna el TTL de las **respuestas negativas**, es decir, cuánto tiempo un resolver puede recordar que un nombre *no existe* (sección 1.8).

La tabla siguiente enumera los tipos de registro que tienen relevancia para el despliegue.

Tipo	Asocia un nombre con...	Uso en este módulo
A	Una dirección IPv4	El que se utiliza. Apunta los subdominios al VPS
AAAA	Una dirección IPv6	Solo si el VPS tiene IPv6 asignada
CNAME	Otro nombre (alias)	Habitual al usar servicios gestionados; acá no hace falta
TXT	Texto arbitrario	Verificación de titularidad ante terceros
NS	Los servidores autoritativos de la zona	Define dónde se cargan los demás registros
MX	El servidor de correo del dominio	Fuera del alcance del módulo
SOA	Los parámetros administrativos de la zona	Lo genera el proveedor; gobierna la caché negativa
CAA	Qué autoridades pueden emitir certificados	Puede impedir la emisión si está mal configurado

1.6.1. Restricciones del registro CNAME

El registro CNAME merece un párrafo conceptual antes de sus restricciones, porque su semántica es distinta de la de todos los demás tipos. Un CNAME no contiene un dato final: declara que un nombre es un **alias** de otro, el nombre canónico. Cuando un resolver encuentra un CNAME, reinicia la consulta con el nombre canónico y repite el proceso hasta llegar a un registro con datos concretos. Es útil cuando el destino real está bajo control de un tercero —un servicio gestionado cuya IP cambia sin aviso—, porque el titular del alias no necesita enterarse de esos cambios. Pero esa misma semántica de sustitución total es la fuente de sus dos limitaciones, que son fuente frecuente de error:

- **No puede coexistir con otros registros en el mismo nombre.** Si un nombre tiene un CNAME, no puede tener además un A, un TXT ni un MX. La razón es lógica, no caprichosa: el CNAME afirma "este nombre es, a todos los efectos, aquel otro"; admitir simultáneamente otros datos propios contradiría esa afirmación y volvería ambigua toda consulta.
- **No puede utilizarse en el vértice de la zona**, es decir, en el dominio raíz pelado (tudominio.com, sin subdominio). El vértice requiere obligatoriamente registros NS y SOA, con los que un CNAME no puede convivir por la regla anterior.

Por esta razón, el dominio raíz siempre se apunta con un registro **A**, nunca con un CNAME. Algunos proveedores ofrecen extensiones propietarias (**ALIAS**, **ANAME**, **CNAME flattening**) que simulan el comportamiento resolviendo el alias del lado del servidor; son válidas, pero no son estándar.

1.6.2. El registro CAA

Un registro CAA (*Certification Authority Authorization*, RFC 8659) declara qué autoridades certificadoras están habilitadas a emitir certificados para el dominio. Su mecánica es preventiva: antes de emitir un certificado, toda autoridad certificadora está obligada a consultar los registros CAA del nombre —ascendiendo por el árbol hasta encontrarlos— y a negarse a emitir si no figura en la lista. Si el dominio tiene un CAA que no incluye a Let's Encrypt, la emisión automática del certificado **falla sin**

explicación evidente: el DNS resuelve perfecto, el servicio funciona, y el único síntoma es un certificado que nunca llega.

⚠ OJO ACÁ

Los dominios recién registrados normalmente no traen CAA, así que esto no te va a molestar. Pero si usás un dominio que ya venías usando —de un proyecto viejo, o del trabajo— y el certificado no sale por más que el DNS esté perfecto, revisá el CAA. Es de los pocos errores que no dejan rastro útil en el log de Traefik.

1.7. Delegación: dónde se cargan los registros realmente

La delegación es el mecanismo que materializa la idea de zona presentada en la sección 1.4: el operador del dominio de primer nivel no almacena los registros de cada dominio registrado, sino únicamente una anotación de **quién** los administra. Esa anotación son los registros NS del dominio, cargados en la zona del padre. Cuando un resolver pregunta por `tudominio.com` al servidor de `.com`, la respuesta no es una dirección: es una referencia —"preguntale a estos servidores de nombres"— y la resolución continúa allí.

Antes de operar con esto conviene distinguir tres roles comerciales que la industria mantiene separados y que el lenguaje cotidiano mezcla. El **registro** (*registry*) es la organización que opera un dominio de primer nivel completo: Verisign opera `.com`, NIC Argentina opera `.ar`. El **registrador** (*registrar*) es la empresa minorista donde el titular compra el dominio: Namecheap, name.com y similares; su función es inscribir el dominio ante el registro y declarar sus servidores de nombres. El **titular** (*registrant*) es quien tiene los derechos sobre el nombre. La consecuencia clave es que el registrador y el **proveedor de DNS** —el que hospeda la zona con los registros— son funciones separables: comprar el dominio en un lugar y administrar sus registros en otro es una configuración perfectamente normal, y esa flexibilidad es precisamente lo que los registros NS expresan.

De ahí la regla operativa central de esta sección: el registrador donde se compra un dominio y el proveedor donde se administran sus registros **no son necesariamente el mismo**. Quien lo determina son los registros NS del dominio, cargados en el operador del dominio de primer nivel. La consecuencia práctica es directa: los registros deben cargarse en el panel del proveedor al que apuntan los NS. Cargarlos en cualquier otro lugar no produce error alguno; simplemente no tiene efecto.

Verificación previa obligatoria antes de cargar cualquier registro:

```
nslookup -type=NS tudominio.com
```

La respuesta indica el proveedor que administra la zona:

Respuesta	Los registros se cargan en
<code>dns1.registrar-servers.com</code>	Namecheap
<code>ns1.dns-parking.com</code>	Hostinger
<code>xxx.ns.cloudflare.com</code>	Cloudflare

Un detalle técnico completa el cuadro: cuando los servidores de nombres de una zona viven **dentro** de la propia zona que delegan —por ejemplo, si los NS de `tudominio.com` fueran

ns1.tudominio.com—, se produce una circularidad: para resolver el NS habría que consultar la zona a la que ese NS da acceso. El protocolo la resuelve con los **registros pegamento** (*glue records*): el padre acompaña la referencia con las direcciones IP de esos servidores. No es una situación que este práctico configure, pero explica por qué algunos paneles de registradores piden "registrar los nameservers con su IP".

⚠ OJO ACÁ

Este es el error número uno de todo el capítulo, y el más frustrante: cargás los registros en el panel de Namecheap, los ves ahí, con todos los datos correctos, y el dominio no resuelve nunca. ¿Por qué? Porque los nameservers estaban delegados a Cloudflare y Namecheap no es autoritativo para esa zona. **El panel te muestra los registros que cargaste, no los que el mundo consulta.** Corré el `nslookup -type=NS` antes de tocar nada.

1.8. Tiempo de vida, caché y el concepto erróneo de "propagación"

El rendimiento del DNS descansa sobre un principio general de los sistemas distribuidos: si un dato se consulta millones de veces y cambia rara vez, lo eficiente es copiarlo cerca de quien pregunta y responder desde la copia. Esa copia local es la **caché**, y como toda caché plantea la misma pregunta inevitable: ¿durante cuánto tiempo es válida la copia? El DNS responde con un contrato explícito: cada registro incluye un valor **TTL** (*Time To Live*), expresado en segundos, que indica durante cuánto tiempo un resolver está autorizado a reutilizar la respuesta sin volver a consultarla.

El TTL es, por lo tanto, una perilla que regula un compromiso. Un TTL alto minimiza consultas al autoritativo y acelera la resolución, al precio de que los cambios tarden en ser vistos; un TTL bajo hace los cambios casi inmediatos, al precio de más tráfico y mayor dependencia de que el autoritativo esté siempre disponible. No existe el valor correcto universal: existe el valor adecuado a la frecuencia de cambio esperada del registro.

Valor de TTL	Equivale a	Uso recomendado
300	5 minutos	Durante una migración o mientras se configura
3600	1 hora	Valor de compromiso habitual
14400	4 horas	Predeterminado en muchos paneles
86400	24 horas	Registros estables que no cambian

La expresión "esperar a que propague" es engañosa: sugiere que la información se difunde activamente por la red, como una onda que avanza de servidor en servidor. **No ocurre nada de eso.** El DNS no tiene ningún mecanismo de difusión: nadie notifica a nadie. Los servidores autoritativos se actualizan de forma prácticamente inmediata; lo que demora es la **expiración de las copias en caché** que los resolvers de todo el mundo obtuvieron antes del cambio. Cada resolver descubre el valor nuevo en un momento distinto: exactamente cuando su copia vieja expira y una consulta lo obliga a volver al autoritativo. La "propagación" no es un proceso: es la suma de millones de expiraciones independientes.

De ahí se desprenden dos consecuencias operativas:

- Un dominio **nuevo**, que nunca fue consultado, resuelve casi de inmediato: no existe caché previa que expirar.
- Un registro **modificado** puede tardar hasta el TTL anterior en reflejarse. Si el TTL era de 24 horas, esa es la espera máxima.

La técnica estándar consiste en **reducir el TTL a 300 segundos con al menos un día de anticipación** al cambio previsto, y restaurarlo una vez verificado el nuevo valor. Nótese la lógica temporal: la reducción del TTL también es un cambio que las cachés descubren al expirar el TTL viejo, por eso debe hacerse con anticipación.

Existe una segunda forma de caché, menos conocida y responsable de un fallo clásico del práctico: la **caché negativa**, normada por la RFC 2308. Los resolvers no solo recuerdan las respuestas afirmativas; también recuerdan los NXDOMAIN, durante un tiempo gobernado por el SOA de la zona. La consecuencia práctica es una trampa frecuente: si se consulta un nombre **antes** de haber creado su registro, el resolver memoriza que no existe, y seguirá respondiendo NXDOMAIN durante varios minutos aunque el registro ya se haya cargado correctamente. Consultar "para ver si ya está" antes de tiempo puede, literalmente, demorar el momento en que se lo ve.

1.8.1. Capas de caché

Una consulta atraviesa hasta cuatro cachés antes de llegar a un servidor autoritativo. Cada capa guarda su propia copia con su propio reloj de expiración, y por eso dos equipos de la misma mesa pueden ver valores distintos: no comparten todas las capas.

Capa	Cómo se vacía
Navegador	<code>chrome://net-internals/#dns</code> → <i>Clear host cache</i>
Sistema operativo	<code>ipconfig /flushdns</code> (Windows) · <code>sudo resolvectl flush-caches</code> (Linux)
Router doméstico	Reiniciar el equipo
Resolver del proveedor	No se puede vaciar; hay que esperar el TTL

OJO ACÁ

Cuando un compañero jure que el dominio no resuelve y a vos te resuelva perfecto, no está mintiendo: está viendo una caché distinta a la tuya. Antes de tocar un solo registro, corré `ipconfig /flushdns` y probá de nuevo. Te vas a ahorrar media hora de diagnóstico de un problema que no existe.

1.9. Estudio de caso: cuando el resolver decide no responder

Las secciones anteriores describieron el sistema funcionando según su diseño. Esta sección estudia un fenómeno real, documentado en cursadas anteriores desde conexiones argentinas, en el que un actor del sistema se aparta deliberadamente del comportamiento esperado. Su valor pedagógico es doble: obliga a aplicar todo el modelo teórico visto hasta acá —resolver, RCODE, jerarquía de autoridad— y demuestra que el DNS es también un punto de control administrativo sobre el tráfico de red.

Easypanel asigna automáticamente un nombre de dominio a todo servicio que no tenga uno propio configurado. Ese nombre es un subdominio de `easypanel1.host`, una zona DNS que la propia empresa administra con sus servidores de nombres `ns1` y `ns2`:

```
app.mi-proyecto.3xz186.easypanel1.host
```

zona administrada por Easypanel

El mecanismo es cómodo: publicado el servicio, el nombre existe y funciona en segundos, sin registrar ningún dominio ni tocar ninguna zona DNS. Existen servicios equivalentes —`sslip.io`, `nip.io`, `traefik.me`— que cumplen la misma función para otras plataformas.

1.9.1. El fallo observado

Desde conexiones de determinados proveedores argentinos, **estos nombres no resuelven**. El navegador informa un error de resolución del tipo `DNS_PROBE_FINISHED_NXDOMAIN` y el servicio resulta inaccesible, pese a estar en perfecto estado de funcionamiento. El mismo nombre, consultado a un resolver público, resuelve sin inconvenientes.

Dos observaciones importantes sobre el alcance del fenómeno:

- **Depende del resolver, no del proveedor.** Un mismo operador puede aplicar el filtrado en su servicio hogareño y no en su red móvil, porque son infraestructuras distintas con configuraciones distintas.
- **No es permanente.** Las listas de bloqueo cambian. Un nombre que hoy no resuelve puede resolver el mes que viene, y al revés.

Las causas documentadas son dos, y pueden presentarse combinadas.

Listas de reputación. Los servicios que reparten subdominios gratuitos e instantáneos son intensamente utilizados para alojar campañas de *phishing* y para infraestructura de control de programas maliciosos, precisamente por ser gratuitos e instantáneos. En consecuencia, la zona completa aparece en listas de reputación a las que muchos resolvers de proveedores están suscriptos, y el bloqueo alcanza por igual a los usos legítimos. Obsérvese la lógica económica del daño colateral: para el operador del resolver, distinguir subdominio por subdominio es costoso, y bloquear la zona entera es barato; los perjudicados —proyectos legítimos sin dominio propio— no son sus clientes directos, así que el incentivo para afinar el filtro es débil.

Protección contra DNS rebinding. Algunos de estos servicios devuelven cualquier dirección que se les solicite, incluidas las de rango privado como `192.168.0.1`. Ese es el vector del ataque conocido como *DNS rebinding*, que merece explicarse porque ilustra un uso ofensivo del protocolo: el navegador aplica una política de mismo origen basada en nombres, no en direcciones; si un atacante logra que un nombre bajo su control resuelva primero hacia su servidor y, minutos después —con un TTL mínimo—, hacia una dirección interna de la red de la víctima, el código que el navegador ya descargó puede dirigir peticiones "al mismo origen" que ahora apuntan al router o a un servicio interno. Numerosos resolvers —y buena parte de los routers domésticos— bloquean como medida preventiva toda zona que devuelva direcciones privadas, y la zona completa paga por el vector.

En ambos casos el síntoma es idéntico y el procedimiento de diagnóstico es el mismo.



Un resolver que **filtra** contesta rápido: NXDOMAIN, REFUSED, o una dirección falsa. Tiene la respuesta lista.

Un resolver que **está caído o inalcanzable** no contesta nada: la consulta agota el tiempo de espera y se reintenta varias veces antes de rendirse.

Las dos cosas se ven parecidas y no son lo mismo. Antes de concluir que hay filtrado, siempre hay que correr la prueba de control de la sección 1.9.2. Un fallo intermitente del resolver, tomado por filtrado, lleva a una explicación técnica perfectamente equivocada de un problema perfectamente real.

1.9.2. Procedimiento de diagnóstico

El diagnóstico requiere **cuatro consultas**, no dos. Las dos primeras son la prueba de control y son las que evitan sacar una conclusión errónea. El método es el experimental clásico: antes de medir el fenómeno hay que verificar el instrumento. La primera consulta (¿resuelve google.com?) comprueba que el resolver está vivo y funcionando; la segunda (¿resuelve la zona de la plataforma?) separa el bloqueo de la zona del bloqueo del nombre puntual; recién la tercera mide el fenómeno, y la cuarta repite la medición con un instrumento distinto —un resolver público— para aislar la variable.

```
nslookup google.com
nslookup easypanel.host
nslookup app.mi-proyecto.3xz186.easypanel.host
nslookup app.mi-proyecto.3xz186.easypanel.host 8.8.8.8
```

Las tres primeras utilizan el resolver que asignó la red; la cuarta fuerza el resolver público de Google. La lectura del resultado es la siguiente:

google.com	La zona de la plataforma	Con 8.8.8.8	Conclusión
Resuelve	No resuelve	Resuelve	Filtrado confirmado. El bloqueo está en el resolver
No resuelve	No resuelve	Resuelve	El resolver está caído o inalcanzable. No hay filtrado
Resuelve	Resuelve	Resuelve	Esta red no aplica filtrado. Probar desde otra

Solo la primera fila autoriza a afirmar que existe filtrado: **el bloqueo ocurre en el resolver, no en el dominio.**

Para identificar qué resolver está respondiendo:

```
ipconfig /all
```

```

C:\Users\Matias Torres>nslookup easypanel.host 223.4.123.234
DNS request timed out.
  timeout was 2 seconds.
Servidor: UnKnown
Address: 223.4.123.234

DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
*** Se agotó el tiempo de espera de la solicitud a UnKnown

C:\Users\Matias Torres>nslookup easypanel.host 8.8.8.8
Servidor: dns.google
Address: 8.8.8.8

Nombre: easypanel.host

C:\Users\Matias Torres>

```

Figura 1.3. La misma consulta, formulada a dos resolvers distintos, produce resultados incompatibles. El filtrado ocurre en el resolver, no en el dominio.

EXPERIMENTO

Hacé esta prueba desde tu casa, con tu propia conexión. Es importante que sea la tuya y no una compartida, y en un momento vas a ver por qué.

1. Ejecutá las cuatro consultas de la sección 1.9.2 y anotá cada salida.
2. Volcá en la planilla compartida del curso **tu proveedor de internet** y **qué te dio cada consulta**.
3. Cuando la planilla esté completa, leela entera.

Vas a encontrarte con que **a algunos compañeros les falló y a otros no**, con la misma consulta y en el mismo momento. Treinta personas en treinta conexiones distintas —Claro, Fibertel, Movistar, Supercanal— son treinta resolvers distintos.

Esa divergencia es todo el punto: el DNS no es una guía telefónica universal, **es un servicio que alguien te presta**, y ese alguien puede filtrarte, mentirte o simplemente no saber. Es el mismo mecanismo por el que un proveedor bloquea un sitio por orden judicial. No hay nada roto: hay alguien que decidió no contestar.

⚠ OJO ACÁ

Si hacés la prueba desde una red compartida, puede no reproducirse. En el wifi de una facultad, una oficina o un bar, todos los equipos salen por el **mismo resolver**. Si ese resolver no aplica el filtrado, las cuatro consultas te van a dar bien y no vas a ver ninguna diferencia.

No significa que el fenómeno no exista: significa que esa red no lo reproduce. Probá **sin wifi, con datos móviles**. Ahí entra el resolver de la compañía de tu celular, y el mismo teléfono, con wifi y sin wifi, te puede dar dos resultados distintos.

1.9.3. Consecuencia para el práctico

Cambiar el resolver del equipo a 8.8.8.8 resuelve el síntoma en esa máquina, pero **no es una solución**: la aplicación quedaría inaccesible para cualquier visitante cuyo proveedor aplique el mismo filtrado. El error de razonamiento que esta observación corrige es sutil y muy común: confundir la propia experiencia de acceso con la accesibilidad del servicio. Quien publica una aplicación no controla

los resolvers de sus visitantes; solo controla la zona que ellos consultarán. La única variable que está del lado del que publica es la reputación del nombre, y un dominio propio, delegado a una zona limpia, es la forma de controlarla.

De aquí se desprende una decisión de diseño del práctico:

DATO

El dominio propio no es un adorno ni un paso opcional del despliegue. Es el único mecanismo que garantiza que la aplicación sea alcanzable con independencia del resolver que use cada visitante. Por eso este capítulo va **primero**, antes de tocar el servidor.

1.10. Herramientas de diagnóstico

La herramienta disponible en todos los sistemas operativos es `nslookup`. En entornos Unix se prefiere `dig`, por ofrecer mayor detalle: su salida reproduce el mensaje DNS casi campo por campo —el RCODE en la primera línea de estado, los indicadores `aa`, `rd` y `ra`, y las cuatro secciones de la sección 1.5—, de modo que leer la salida de `dig` es, en la práctica, leer el protocolo. Conviene acostumbrarse a buscar dos datos en cada salida: el **status** (¿NOERROR, NXDOMAIN, SERVFAIL?) y **quién respondió** (la línea `SERVER`, que identifica al resolver consultado).

Objetivo	Comando
Resolver un nombre	<code>nslookup calculadora.tudominio.com</code>
Consultar un resolver específico	<code>nslookup calculadora.tudominio.com 8.8.8.8</code>
Ver los servidores autoritativos	<code>nslookup -type=NS tudominio.com</code>
Ver registros de texto	<code>nslookup -type=TXT tudominio.com</code>
Ver registros CAA	<code>nslookup -type=CAA tudominio.com</code>
Detalle completo (Unix)	<code>dig calculadora.tudominio.com +noall +answer</code>
Traza completa de la jerarquía (Unix)	<code>dig calculadora.tudominio.com +trace</code>

El modificador `+trace` de `dig` merece mención especial por su valor didáctico: en lugar de preguntarle al resolver, reproduce él mismo la resolución iterativa completa de la sección 1.4.3 —raíz, TLD, autoritativo— mostrando cada referencia recibida. Es la forma más directa de ver la jerarquía trabajando, y también la forma más limpia de esquivar todas las cachés intermedias cuando se sospecha de ellas.

Para verificar la resolución desde múltiples ubicaciones geográficas simultáneamente se utiliza `dnschecker.org`, que consulta decenas de resolvers distribuidos por el mundo y presenta el resultado en un mapa.

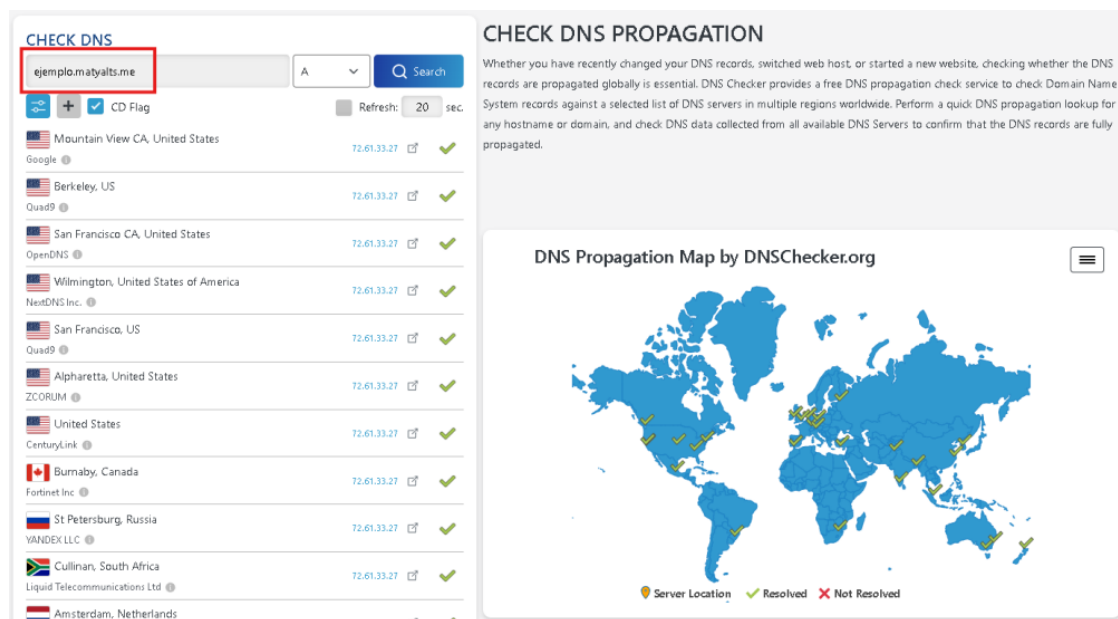


Figura 1.4. Verificación de la resolución desde múltiples ubicaciones geográficas.

PARA ENTENDER

nslookup te dice qué contesta **un** resolver. dnschecker.org te dice qué contestan cincuenta. Cuando el dominio te anda a vos y no le anda a medio curso, esa diferencia es todo el diagnóstico.

Obsérvese en la figura anterior un detalle que no es un error de la herramienta: entre decenas de resolvers que devuelven la dirección correcta, **alguno puede no devolver nada**. Las causas posibles son varias —filtrado, una caché negativa anterior a la creación del registro, o simplemente ese resolver fuera de servicio en ese instante— y la herramienta no permite distinguir entre ellas.

Lo que sí demuestra, y es el punto: **no todos los resolvers del mundo responden lo mismo en el mismo momento**. Un dominio recién configurado que funciona desde una conexión puede no funcionar desde otra, sin que exista nada mal en el dominio ni en el servidor.

OJO ACÁ

Y de acá sale la regla operativa: **"a mí me anda" no es una verificación**. Es una muestra de tamaño uno, tomada desde un solo resolver, con una sola caché. El día que vos digas que tu sitio anda y tres compañeros digan que no, los cuatro van a estar diciendo la verdad.

1.11. Seguridad y evolución del protocolo: DNSSEC, DoT y DoH

El DNS de 1983 se diseñó para una red de instituciones que confiaban entre sí, y por eso carece de dos propiedades que hoy se consideran elementales: las respuestas no están autenticadas —nada prueba que provengan realmente del autoritativo— y viajan en texto plano —cualquier intermediario puede leerlas—. El estudio de caso de la sección 1.9 mostró la consecuencia práctica de esa arquitectura: el resolver es un intermediario con poder total sobre lo que el usuario ve, y puede filtrar o mentir sin que el protocolo original ofrezca forma de detectarlo. Las tres extensiones de esta sección atacan esas carencias, cada una desde un ángulo distinto, y conocerlas —aunque el práctico no las configure— completa el mapa conceptual del sistema.

1.11.1. DNSSEC: autenticar las respuestas

DNSSEC (*DNS Security Extensions*) agrega al DNS firmas criptográficas. El titular de una zona firma sus registros con una clave privada y publica las firmas (registros `RRSIG`) junto con la clave pública (registro `DNSKEY`) como registros más de la zona. Un resolver que valida puede entonces comprobar matemáticamente que la respuesta recibida es la que el autoritativo publicó, sin alteraciones.

La pregunta obvia es cómo se confía en la clave pública misma, y la respuesta reutiliza elegantemente la estructura que ya se estudió: la jerarquía. La zona padre publica un resumen criptográfico de la clave de su zona hija (registro `DS`), de modo que la confianza se encadena: la clave de `tudominio.com` está avalada por `.com`, la de `.com` por la raíz, y la clave de la raíz —firmada desde 2010— es el único punto de partida que los resolvers deben conocer de antemano. Es la misma cadena de delegación de la sección 1.7, duplicada en el plano criptográfico. Cuando la validación falla —una firma vencida, una cadena rota—, el resolver validante responde `SERVFAIL`: otro de los significados posibles de ese código que la sección 1.5 dejó anotado.

Dos precisiones delimitan qué es y qué no es DNSSEC. Primero: **auténtica, no cifra**; las consultas siguen siendo visibles para cualquier observador de la red. Segundo: no impide el filtrado del estudio de caso —un resolver siempre puede negarse a responder—, pero sí impide la variante más grave, la respuesta falsificada, en zonas firmadas y ante resolvers que validan.

1.11.2. DoT y DoH: cifrar el canal

El segundo problema —la visibilidad del tráfico— se ataca cifrando el tramo entre el equipo del usuario y el resolver. Existen dos estándares con el mismo propósito y distinto empaque: **DNS over TLS** (DoT, RFC 7858) envuelve las consultas en un canal TLS sobre el puerto dedicado 853, mientras que **DNS over HTTPS** (DoH, RFC 8484) las envía como tráfico HTTPS común por el puerto 443, indistinguible de la navegación web. La diferencia de puerto no es un detalle menor: el tráfico DoT es identificable (y bloqueable) como tráfico DNS, mientras que bloquear DoH implicaría bloquear HTTPS entero. Los navegadores modernos incorporan DoH y pueden activarlo por defecto hacia resolvers públicos.

Conviene ubicar con precisión qué protegen: cifran **el canal hasta el resolver**, no la resolución completa, y el resolver elegido sigue viendo todas las consultas. En términos del estudio de caso: un estudiante con DoH activo hacia `1.1.1.1` esquivo el filtrado de su proveedor —su tráfico DNS ya no pasa por el resolver que filtra—, lo que explica por qué "a algunos compañeros les anda" incluso dentro de la misma red y proveedor. Pero la conclusión de la sección 1.9.3 no cambia: el que publica un servicio no puede exigirle DoH a sus visitantes. La solución del lado del publicador sigue siendo el dominio propio.

DATO

Estas tecnologías redistribuyen la confianza, no la eliminan. Sin DoH, tu proveedor de internet ve tus consultas; con DoH hacia un resolver público, las ve ese resolver. La pregunta "¿quién ve mis consultas DNS?" nunca tiene respuesta "nadie": tiene respuesta "el que vos elegiste como resolver". Elegir resolver es elegir en quién confiar.

1.12. Registros comodín: dos conceptos distintos

La palabra *wildcard* designa dos cosas diferentes en el contexto del despliegue. Confundirlas es habitual y lleva a detenerse ante una complejidad que en este práctico no hace falta enfrentar.

	Registro DNS comodín	Certificado TLS comodín
Qué es	Un registro A con nombre *	Un certificado válido para *.dominio.com
Para qué sirve	Que cualquier subdominio resuelva	Que cualquier subdominio tenga HTTPS
Cómo se obtiene	Se carga en el panel DNS	Requiere validación DNS-01
Qué necesita	Nada más	Un token de API del proveedor DNS, configurado en Traefik
Dificultad	Trivial	Considerable
¿Se usa en este módulo?	Sí	No

1.12.1. El registro comodín

El comodín DNS está definido desde la RFC 1034 (y precisado en la RFC 4592) con una semántica exacta: un registro cuyo nombre es * actúa como respuesta de **último recurso** para los nombres que no existen en la zona. La sutileza está en la condición: el comodín solo se sintetiza cuando la consulta no encuentra ninguna coincidencia explícita. No es un "valor por defecto" que se mezcla con lo existente; es lo que la zona responde cuando no tiene nada más específico que decir.

Un único registro comodín reemplaza la necesidad de crear un registro por cada subdominio:

Tipo	Nombre	Apunta a	Efecto
A	*	IP del VPS	Cualquier subdominio resuelve al VPS
A	@	IP del VPS	El dominio raíz pelado también resuelve

Con esos dos registros, `calculadora.tudominio.com`, `api.tudominio.com`, `easypanel.tudominio.com` y cualquier otro nombre que se invente en el futuro resuelven sin necesidad de volver a tocar el panel DNS.

Dos precisiones sobre el alcance del comodín:

- **No cubre el dominio raíz.** *.tudominio.com no incluye a tudominio.com. Por eso se agrega el registro con nombre @.
- **Un registro explícito tiene prioridad.** Si más adelante se crea un registro A para api apuntando a otra dirección, ese registro gana sobre el comodín. El comodín actúa únicamente cuando no existe una coincidencia más específica.

Vale notar una asimetría que a veces sorprende: el comodín DNS cubre también nombres de varios niveles (`a.b.tudominio.com` resuelve, si ningún nombre intermedio existe explícitamente), mientras que el certificado comodín *.dominio.com cubre **un solo nivel** de etiqueta. Son reglas de sistemas distintos que comparten el mismo símbolo.

1.12.2. Por qué no hace falta el certificado comodín

Cada servicio publicado en Easypanel solicita **su propio certificado individual** mediante el desafío HTTP-01 de Let's Encrypt, que se valida sirviendo un archivo por el puerto 80. Ese mecanismo es automático y no requiere configuración adicional. El certificado comodín, en cambio, exige el desafío DNS-01 —demostrar control sobre la zona creando un registro TXT al vuelo—, lo que requiere darle a Traefik credenciales de API del proveedor DNS: más piezas, más secretos que custodiar, más puntos de fallo.

El certificado comodín solo sería necesario para cubrir subdominios que nunca se publican individualmente, un escenario que este práctico no presenta.

⚠ OJO ACÁ

La documentación oficial de Easypanel sobre wildcard domains habla de configurar un *certificate resolver* con credenciales de API de tu proveedor DNS. Es correcto, pero es para el **certificado comodín**, no para el registro. Si la lees y te frenás pensando que necesitás todo eso, quedate tranquilo: con el registro * tipo A alcanza y sobra. Es la confusión más común de esta clase.

1.13. Obtención del dominio mediante el GitHub Student Developer Pack

El GitHub Student Developer Pack es un conjunto de beneficios para estudiantes con matrícula vigente. Entre ellos se incluye el registro gratuito de un dominio por el término de un año.

Proveedor	Oferta	Observaciones
Namecheap	Un dominio <code>.me</code> gratuito por un año	Incluye certificado SSL y privacidad de WHOIS
name.com	Un dominio gratuito por un año	Más de 25 extensiones: <code>.live</code> , <code>.studio</code> , <code>.software</code> , <code>.app</code> , <code>.dev</code>
.TECH	Un dominio <code>.tech</code> gratuito por un año	Extensión única

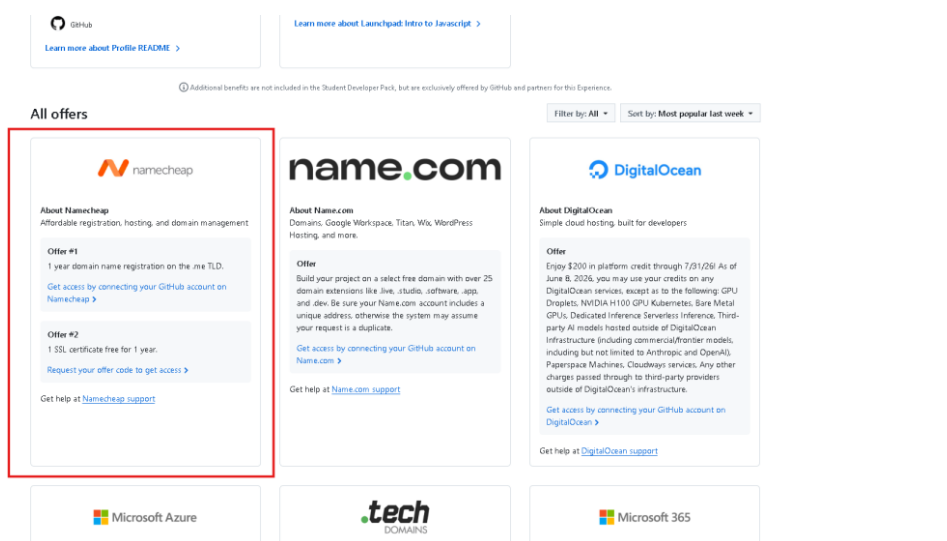


Figura 1.5. Ofertas de registro de dominio incluidas en el GitHub Student Developer Pack.

Recomendación para el práctico: Namecheap con un dominio `.me`. Es la opción con el panel DNS más simple y la que menos pasos intermedios requiere.

 DATO

Las extensiones .app y .dev de name.com están en la lista de precarga HSTS de los navegadores. Eso significa que el navegador **exige HTTPS obligatoriamente**: no existe la posibilidad de acceder por http://. Es una buena práctica impuesta por diseño, pero conviene saberlo de antemano para no diagnosticar mal un fallo durante la Clase 4.

1.13.1. Verificación de la condición de estudiante

El trámite exige acreditar la matrícula vigente mediante alguno de estos elementos:

- Certificado de alumno regular emitido por la Facultad.
- Constancia de inscripción a materias del ciclo lectivo en curso.
- Credencial estudiantil con fecha de vencimiento visible.

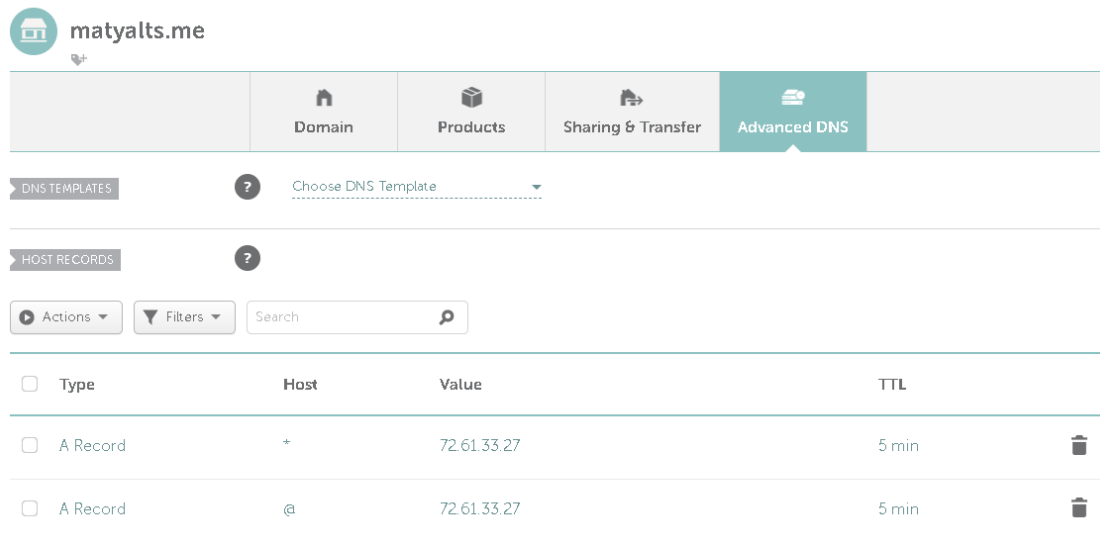
 OJO ACÁ

Este trámite tarda **entre 3 días y 3 semanas**, y en un porcentaje nada despreciable de casos lo rechazan y hay que rehacerlo. Arrancalo con cuatro semanas de anticipación, no el día antes de necesitar el dominio. El instructivo con los errores típicos del rechazo está en la hoja de tarea previa.

1.14. Carga de los registros

Una vez obtenido el dominio y verificada la delegación (sección 1.7), se cargan dos registros en el panel del proveedor autoritativo. El procedimiento es la síntesis operativa de todo lo anterior: el comodín aplica la semántica de la sección 1.12, el registro de vértice cubre la limitación que el comodín no alcanza, y el TTL bajo aplica el análisis de la sección 1.8 a un contexto —un práctico donde se corrige y reintenta— en el que la frecuencia de cambio esperada es alta.

Campo	Registro comodín	Registro raíz
Tipo	A	A
Nombre / Host	*	@
Valor / Points to	Dirección IP del VPS	Dirección IP del VPS
TTL	300 durante el práctico	300 durante el práctico



Type	Host	Value	TTL
A Record	*	72.61.33.27	5 min
A Record	@	72.61.33.27	5 min

Figura 1.6. Registro comodín y registro del vértice apuntando a la dirección del VPS.

⚠ OJO ACÁ

Tres errores que te van a aparecer sí o sí en la clase:

- 1. Escribir el nombre completo en el campo Nombre.** Va *, no *.tudominio.com. El panel agrega el dominio solo. Si ponés el nombre completo terminás con *.tudominio.com.tudominio.com, que no existe y no resuelve nunca.
- 2. Confundir @ con el correo.** En una zona DNS, @ significa "el dominio raíz, sin subdominio". No tiene absolutamente nada que ver con el email.
- 3. Dejar el TTL en 4 horas.** Durante el práctico se equivocan y corrigen varias veces. Con TTL alto, cada error cuesta media clase de espera. Ponelo en el mínimo.

🔗 DATO

Los paneles no siempre piden el TTL en segundos. **Namecheap lo ofrece como una lista desplegable** con opciones del tipo *Automatic, 1 min, 5 min, 30 min*: los 300 segundos que pide este práctico son la opción **"5 min"**. Otros proveedores sí piden el número. Es la misma magnitud expresada de dos maneras, pero si buscás dónde tipear "300" en Namecheap no lo vas a encontrar nunca.

En la sección *Advanced DNS* del dominio, el panel de Namecheap presenta los registros en una tabla con las columnas **Type**, **Host**, **Value** y **TTL**. La columna *Host* es la que en este capítulo se denomina "Nombre".

En la Clase 2 se verificará que la dirección IP cargada corresponde efectivamente al VPS provisionado.

1.15. Verificación

El capítulo se da por cerrado cuando las cuatro comprobaciones siguientes son satisfactorias. Obsérvese que no son cuatro variantes de la misma prueba: cada una valida una pieza distinta del modelo teórico. La primera confirma la delegación (sección 1.7); la segunda, el registro de vértice; la tercera, la semántica del comodín (sección 1.12); y la cuarta, la coherencia entre resolvers independientes, que es la única defensa contra el efecto "a mí me anda" (sección 1.10).

#	Comprobación	Comando	Resultado esperado
1	Delegación correcta	<code>nslookup -type=NS tudominio.com</code>	Los NS del proveedor donde se cargaron los registros
2	Resolución del dominio raíz	<code>nslookup tudominio.com</code>	La IP del VPS
3	Resolución de un subdominio arbitrario	<code>nslookup loquesea.tudominio.com</code>	La misma IP, por efecto del comodín
4	Coherencia entre resolvers	<code>nslookup loquesea.tudominio.com 8.8.8.8</code>	La misma IP que en el punto 3

La comprobación 3 es la que valida el registro comodín: el nombre `loquesea` no fue cargado en ninguna parte y, sin embargo, debe resolver.

⚠ OJO ACÁ

No se avanza a la Clase 4 sin estas cuatro comprobaciones en verde. Let's Encrypt valida la titularidad del dominio conectándose a él: si el nombre no resuelve hacia el servidor, el certificado no se emite, y vas a pasar la clase entera peleando con Easypanel cuando el problema está acá.

1.16. Errores frecuentes

La tabla siguiente condensa los fallos observados en cursadas anteriores. Vale la pena leerla con el modelo teórico en mente: cada síntoma es la manifestación de un concepto de este capítulo — delegación, caché, comodín, caché negativa, CAA— y el diagnóstico correcto consiste en reconocer cuál.

Síntoma	Causa	Resolución
El registro está cargado y el dominio no resuelve	Los NS están delegados a otro proveedor	Verificar con <code>nslookup -type=NS</code> y cargar donde corresponde
El subdominio resuelve concatenado dos veces	Se escribió el nombre completo en el campo Nombre	Dejar solo *, api o @
Resuelve en un equipo y no en otro	Caché local o del proveedor	<code>ipconfig /flushdns</code> y reintentar
El dominio raíz no resuelve pero los subdominios sí	Falta el registro con nombre @	Agregar el registro A del vértice
Un dominio autogenerado no resuelve	Filtrado del resolver (sección 1.9)	Usar dominio propio
El cambio no se refleja después de horas	TTL anterior muy alto	Esperar el TTL previo; bajarlo antes del próximo cambio
Un registro recién creado da NXDOMAIN un rato largo	Caché negativa: se consultó el nombre antes de crearlo (sección 1.8)	Esperar el TTL negativo o probar con otro resolver

Síntoma	Causa	Resolución
El certificado no se emite pese a resolver bien	Registro CAA restrictivo	Consultar con <code>nslookup -type=CAA</code>

1.17. Actividades

Actividad 1 — Trazado de la jerarquía.

Ejecutar `nslookup -type=NS` sobre tres dominios: el propio, `utn.edu.ar` y `github.com`. Identificar el proveedor DNS de cada uno y justificar la respuesta. Quienes dispongan de un entorno Unix pueden repetir el ejercicio con `dig +trace` sobre el dominio propio e identificar en la salida los tres niveles de la sección 1.4.2 y las referencias de la resolución iterativa.

Actividad 2 — Relevamiento colectivo del filtrado.

Ejecutar las cuatro consultas de la sección 1.9.2 —incluida la prueba de control— desde la conexión hogareña propia. Volcar el resultado en la planilla compartida del curso, con estas columnas:

Proveedor	Localidad	Resolver por defecto	¿Resolvió?	¿Resolvió con 8.8.8.8?

Una vez completa la planilla, responder por escrito:

1. ¿Qué proveedores aplican el filtrado y cuáles no?
2. ¿Por qué la segunda consulta funciona en todos los casos?
3. Si la aplicación quedara publicada en un subdominio autogenerado de la plataforma, ¿qué porcentaje del curso no podría acceder? ¿Qué implica eso para un sitio con usuarios reales?

Actividad 3 — Medición del efecto del TTL.

Modificar el registro comodín para que apunte a `1.1.1.1`, verificar el cambio, restaurarlo a la IP del VPS y medir cuánto tarda cada transición en reflejarse. Relacionar el resultado con el valor de TTL configurado.

Actividad 4 — Verificación del comodín.

Comprobar que tres subdominios inventados en el momento resuelven a la IP del VPS sin haber sido cargados individualmente. Comprobar además que un subdominio de dos niveles (`a.b.tudominio.com`) también resuelve, y explicar por qué, según la semántica de la sección 1.12.1.

Actividad 5 — Verificación global.

Consultar el dominio propio en `dnschecker.org` y registrar desde cuántas ubicaciones resuelve correctamente.

Actividad 6 — Lectura del mensaje DNS. (requiere entorno Unix)

Ejecutar `dig tudominio.com` y localizar en la salida: el RCODE (`status`), los indicadores `rd` y `ra`, y las secciones de pregunta y respuesta. Ejecutar luego `dig noexiste-xyz.tudominio.com.invalid` y

comparar el RCODE obtenido. Finalmente, consultar directamente a uno de los servidores autoritativos del dominio (`dig tudominio.com @ns1...`) y verificar la presencia del indicador `aa`, explicando su significado según la sección 1.5.

Actividad 7 — Exploración de DNSSEC. (requiere entorno Unix)

Ejecutar `dig cloudflare.com +dnssec` contra el resolver `1.1.1.1` y observar el indicador `ad` (*authenticated data*) y los registros RRSIG de la respuesta. Repetir con `github.com` y comparar. ¿Cuál de las dos zonas está firmada? ¿Qué protección concreta tiene la primera que la segunda no, según la sección 1.11.1?

1.18. Síntesis

1. El DNS reemplazó a un archivo centralizado que no escalaba. Sus tres decisiones de diseño — autoridad distribuida, caché con coherencia eventual y registros tipados extensibles— explican todo el comportamiento observable en este práctico.
2. La resolución de nombres es **previa e independiente** de la conexión al servidor. Cuando falla, el servidor no registra nada.
3. El sistema de nombres es una **jerarquía distribuida** de zonas separadas por delegaciones, y el usuario nunca la consulta directamente: siempre lo hace a través de un **resolver**, que puede cachear, fallar o filtrar.
4. El mensaje DNS lleva códigos de respuesta que son información de diagnóstico: **NXDOMAIN es una respuesta; un timeout no lo es**. Distinguirlos separa un filtrado de un resolver caído.
5. Los registros se cargan **donde apuntan los NS**, no necesariamente donde se compró el dominio.
6. La "propagación" no existe como tal: lo que ocurre es la **expiración de cachés**, gobernada por el TTL. También lo que no existe se cachea: la **caché negativa** castiga consultar un nombre antes de crearlo.
7. Los dominios autogenerados por Easypanel **no son confiables en Argentina**. El dominio propio es un requisito, no una mejora.
8. DNSSEC autentica las respuestas mediante una cadena de confianza que replica la jerarquía de delegación; DoT y DoH cifran el canal hasta el resolver. Ninguna de las tres elimina al intermediario: redistribuyen la confianza.
9. Un **registro** comodín es trivial y resuelve el problema del práctico. Un **certificado** comodín es otra cosa y no hace falta.

1.19. Referencias y lecturas complementarias

Las fuentes normativas del protocolo son las RFC del IETF, todas de acceso libre en [rfc-editor.org](https://www.rfc-editor.org). Las fundacionales son la RFC 1034 (*Domain Names — Concepts and Facilities*) y la RFC 1035 (*Domain Names — Implementation and Specification*), ambas de P. Mockapetris, 1987; de ellas provienen el modelo de árbol, las zonas, los registros de recursos y el comodín, cuya semántica precisa la RFC 4592. La caché negativa está normada por la RFC 2308; la extensión EDNS(0) por la RFC 6891; el registro CAA por la RFC 8659. Para las extensiones de seguridad: la RFC 9364 ofrece la visión de conjunto de DNSSEC, la RFC 7858 define DNS over TLS y la RFC 8484 define DNS over HTTPS.

Como bibliografía de estudio, el capítulo de capa de aplicación de Kurose y Ross, *Computer Networking: A Top-Down Approach* (8.ª edición, Pearson, 2021) presenta el DNS con el mismo enfoque descendente de este capítulo, y Tanenbaum, Feamster y Wetherall, *Computer Networks* (6.ª edición, Pearson, 2021) lo trata dentro de la capa de aplicación con mayor detalle de protocolo. La referencia operativa clásica, orientada a administradores, es Liu y Albitz, *DNS and BIND* (5.ª edición, O'Reilly, 2006), que pese a su edad sigue siendo la descripción más completa del funcionamiento de zonas y delegaciones.